

Tutorial To Product Plan

This plan keeps the tutorial intact first, then layers CodeBounty-specific features after the foundation is working.

Phase 0: Project Direction

Product identity:

- Name: CodeBounty
- Positioning: pattern-first DSA practice platform with user-hosted coding sessions
- Stack: Next.js, TypeScript, Shadcn UI, Tailwind CSS, Clerk, Prisma, PostgreSQL, Judge0, Gemini

Build rule:

Follow the tutorial as-is until the base platform works. Do not customize core architecture mid-course.

Phase 1: Tutorial Foundation

1. Setup

Tutorial files:

- `02. Project Setup - Installing Nextjs and Initializing Shadcnui`

Expected output:

- Next.js app
- TypeScript setup
- Tailwind and Shadcn UI
- base layout
- theme support

Custom change:

- Only use the product name CodeBounty in visible brand text.
- Do not change architecture yet.

2. Database

Tutorial files:

- 03. Database Setup Made Easy Prisma Initialization & Dockerized DB for Next.js

Expected output:

- Prisma initialized
- PostgreSQL database connected
- Docker database flow understood

Custom change:

- None during tutorial.

3. Authentication And User Onboarding

Tutorial files:

- 04. Authentication Using Clerk -Clerk Setup, GitHub OAuth & Custom Auth Pages
- 05. User Onboarding with Prisma - Defining Schema & Server Actions

Expected output:

- Clerk sign in/sign up
- GitHub OAuth
- user persisted in database

Custom change:

- Keep role support simple because session hosting should not depend on being a teacher.
- Prefer roles: `USER` , `ADMIN` .
- Every signed-in user should be able to create and join sessions.
- Admin-only permissions should be reserved for platform moderation and official problem management.

4. Navbar, Homepage, Role UI

Tutorial files:

- 06. Building a Responsive Navbar & Homepage with DarkLight Mode + Role-Based Auth

Expected output:

- responsive navigation
- home page
- dark/light mode
- role-based links

Custom change:

- Keep homepage simple.
- Later, replace generic clone messaging with pattern-first DSA positioning.

5. Judge0 Setup

Tutorial files:

- `07. Judge0 Setup Self-Hosting Locally or Cloud Integration via RapidAPI`

Expected output:

- Judge0 available locally or through API
- environment variables configured
- code execution service ready

Custom change:

- None during tutorial.

6. Problem Creation

Tutorial files:

- `08. Create Problem Defining Prisma Schema & Linking with Admin User`
- `09. Problem Creation - Integrating Judge0 to Validate & Save Testcases in DB`
- `10. Role-Based Access & UI for Problem Creation Form with Sample Problems`

Expected output:

- problem schema
- admin problem creation
- Judge0 validation before saving
- sample problems

Custom change after tutorial:

- Add `Pattern` , `Tag` , and pattern relation fields.
- Update problem form to include primary pattern and tags.
- Seed sample problems by pattern.

7. Problem Listing

Tutorial files:

- `11. Problem Route Page Fetching All Problems with Table UI, Filters & Search`

Expected output:

- problems route
- table UI
- filters and search

Custom change after tutorial:

- Add pattern filter.
- Add solved/unsolved filter.
- Add "Recommended order" sorting for a selected pattern.

8. Problem Solving

Tutorial files:

- 12. Problem Solving Page Fetch by ID with Header, LeftRight Sections & Full Layout
- 13. Submissions & TestCases Prisma Schema for Storing Execution Data

Expected output:

- problem detail page
- editor and execution layout
- run and submit behavior
- submission storage
- testcase result storage

Custom change after tutorial:

- Add AI hint panel.
- Add session context support to submissions.
- Keep hints disabled during sessions when session policy says so.

9. Submission History

Tutorial files:

- 14. Submission History Fetch User-Specific Submissions & Build History Component

Expected output:

- user-specific submissions
- submissions tab/component

Custom change after tutorial:

- Show pattern information in submission history.

- Show whether submission happened inside normal practice or session mode.

10. Playlists

Tutorial files:

- 15. Prisma Schema & Backend Endpoints for Creating Playlists and Adding Problems
- 16. Create Playlist & Add Problems via Modal with Proper Logic Handling

Expected output:

- playlist schema
- create playlist
- add/remove problems from playlist

Custom change after tutorial:

- Rebrand playlists as sheets where useful.
- Add pattern-aware quick add.

11. Profile

Tutorial files:

- 17. Profile Page Fetch User Info, Stats, Submissions & Playlists

Expected output:

- user profile
- stats
- submissions
- playlists

Custom change after tutorial:

- Add pattern progress heatmap.
- Add session attempt history.
- Add weak pattern recommendations.

Phase 2: Pattern System

Goal:

Make CodeBounty pattern-first.

Implementation steps:

1. Add `Pattern` , `Tag` , and `ProblemTag` models.
2. Add `primaryPatternId` to `Problem` .
3. Create seed data for common DSA patterns.
4. Update problem create/edit form.
5. Update problem list filters.
6. Add `/patterns` page.
7. Add `/patterns/[slug]` page.
8. Update profile stats to group solved problems by pattern.

Acceptance criteria:

- Every problem can have a primary pattern.
- Users can browse and filter by pattern.
- Profile shows solved count per pattern.

Phase 3: AI Hint Button

Goal:

Help users when stuck without destroying learning.

Implementation steps:

1. Add Gemini API environment variable.
2. Create server action or route handler for hints.
3. Add prompt template with progressive hint levels.
4. Store generated hints in `AIHint` .
5. Add hint button/panel on problem solving page.
6. Add basic per-user rate limiting.
7. Disable hints in session mode when configured.

Prompt behavior:

- Do not provide final code unless the requested hint level allows pseudocode.
- Teach the pattern and key observation.
- Use the problem's constraints and examples.
- Keep hints short and actionable.

Acceptance criteria:

- User can request hint levels 1 to 5.
- Hints are relevant to the current problem.
- Repeated same-level requests reuse stored hints.

Phase 4: User-Hosted Session System

Goal:

Add the collaborative workflow that makes the project useful for teachers, friend groups, coding clubs, and practice batches without forcing fixed roles.

Implementation steps:

1. Add `Session`, `SessionProblem`, `SessionParticipant`, and `SessionAttempt` models.
2. Add `/sessions` dashboard route showing hosted and joined sessions.
3. Add session creation form.
4. Allow selecting problems from the existing problem table.
5. Generate a short `joinCode` and store a hashed session password.
6. Add `/sessions/join` flow for ID and password entry.
7. Add session attempt route for participants.
8. Store submissions with `sessionAttemptId`.
9. Add host results page and leaderboard.

Acceptance criteria:

- Any signed-in user can create a session from existing problems.
- Another user can join the session with ID and password.
- Participants can attempt session problems during the valid time window.
- Host can see participant results and leaderboard.
- Session hint policy is respected.

Phase 5: Profile And Analytics Upgrade

Goal:

Make progress visible and motivating.

Implementation steps:

1. Compute solved stats by pattern.
2. Add heatmap or grid component.
3. Add difficulty distribution.
4. Add recent session attempts.
5. Add "continue practice" recommendations.

Acceptance criteria:

- Profile clearly shows strengths and weak patterns.

- User can navigate from a weak pattern to recommended problems.

Phase 6: Resume Polish

Goal:

Make the final project readable and impressive to reviewers.

Implementation steps:

1. Rename app fully to CodeBounty.
2. Write strong README.
3. Add screenshots.
4. Add seeded demo data.
5. Add architecture section.
6. Add feature list and future roadmap.
7. Deploy if possible.

Resume pitch:

“Built CodeBounty, a pattern-first DSA learning platform with Judge0-powered code execution, Clerk authentication, Prisma/PostgreSQL data modeling, AI-powered progressive hints, user-hosted timed coding sessions, leaderboards, and pattern-wise progress analytics.”

Recommended Build Order

1. Complete tutorial without custom changes.
2. Commit tutorial base.
3. Add pattern schema and filters.
4. Add pattern pages and progress stats.
5. Add AI hints.
6. Add user-hosted sessions.
7. Polish profile analytics.
8. Write README and prepare demo.

What Not To Change Early

- Do not change Judge0 flow before it works.
- Do not redesign the problem solving page before submissions work.
- Do not add AI before problem data and execution are stable.
- Do not build sessions before normal submissions work.
- Do not overbuild classroom management in v1.

Scalable Session Architecture

The session system should be modeled as a flexible product primitive instead of a teacher-only feature.

Core idea:

- Admin manages the official problem bank and platform safety.
- User practices problems, creates playlists, joins sessions, and hosts sessions.
- Session ownership controls host actions.
- Join code and password control participant access.
- Timing controls when participants can solve.
- Visibility controls whether leaderboard/results are public to participants or host-only.

Session use cases:

- A teacher creates a college quiz.
- A friend creates a weekend challenge.
- A coding club creates a practice contest.
- A senior creates a placement prep mock round.
- A learner creates a private timed drill for self-practice.

Recommended session types:

- **PRACTICE** : relaxed group practice, optional leaderboard.
- **QUIZ** : time-bound assessment, host-focused results.
- **CONTEST** : competitive leaderboard, stricter timing.
- **MOCK_INTERVIEW** : small problem set with score and review.

Recommended scoring v1:

- Accepted submission earns problem points.
- Earlier accepted submissions rank higher as tie-breaker.
- Wrong submissions can be tracked but should not need penalty in MVP.
- Leaderboard sorts by score, solved count, then last accepted submission time.